Earnings Call Sentiment Analyzer — Design Doc

Nov 6, 2026 · @lenny

Overview

Earnings Call Sentiment Analyzer scores what executives and analysts say on quarterly earnings calls, then tests whether that tone predicts the stock's move over the next 1 to 5 trading days. Users pick a ticker, see its price history with each earnings call marked and colored by sentiment, and click into any call to read the sentences that drove the score.

The project combines NLP (a transformer sentiment model), a classical ML layer (predicting post-earnings returns), and an interactive web app. Every component uses free data, free open-source models, and free hosting.

Goals and non-goals

The project succeeds if it answers one question honestly: does earnings-call tone carry information about the next few days of returns, beyond what the headline numbers already explain?

Goals

- Score every sentence of a transcript for sentiment, and roll scores up by section (prepared remarks vs. analyst Q&A) and by speaker (CEO, CFO, analysts).
- Build a model that predicts post-call abnormal return (stock return minus S&P 500 return) over 1, 3 and 5 days.
- Report results against a naive baseline, with out-of-time backtests, so the claims are defensible in an interview.
- Ship a public web app where anyone can explore a ticker in under 30 seconds.
- Spend $0: free data, open-source models, free hosting.

Non-goals

- Not a trading system or investment advice; the app says so on every page.
- No real-time, minute-by-minute data; daily prices are enough.
- No user accounts, saved portfolios or payments.
- No training a language model from scratch; fine-tune or use pre-trained models only.

User experience

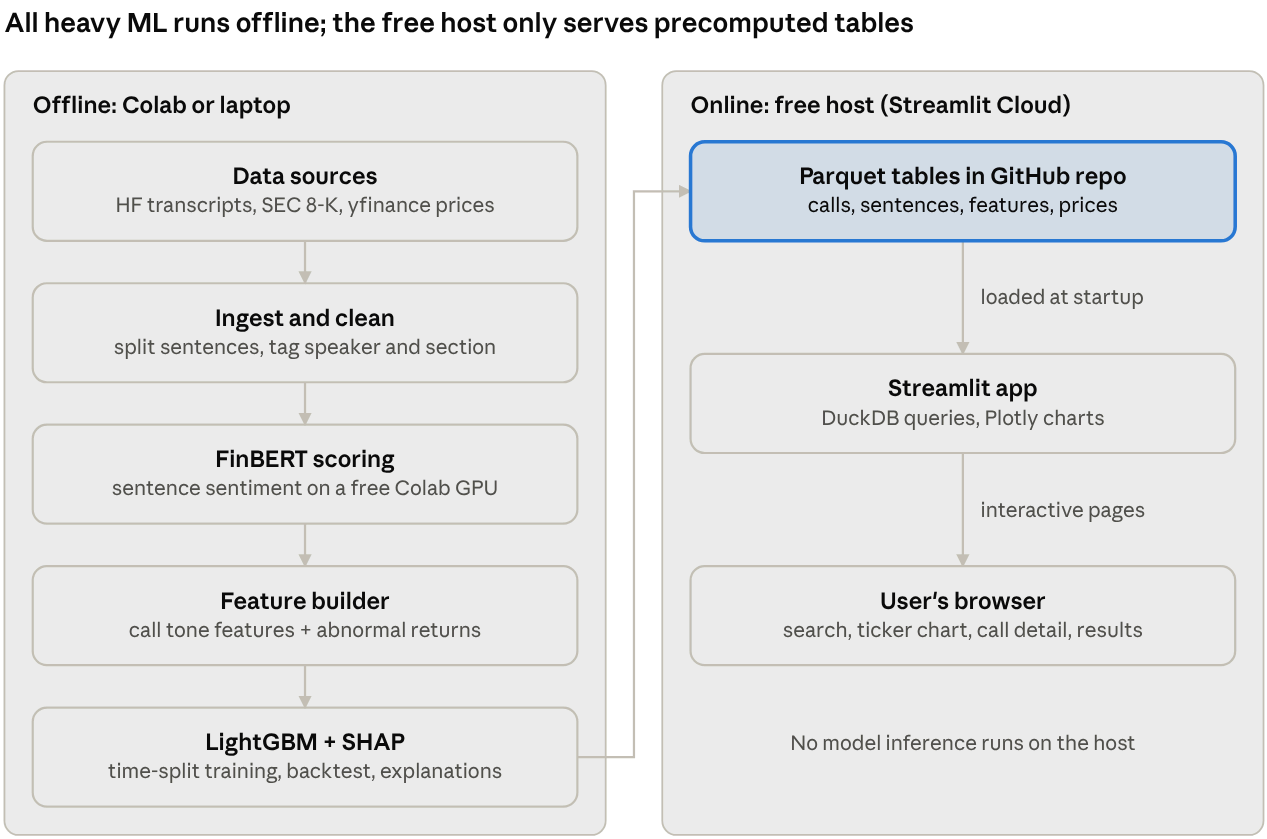
The app has four screens, and the ticker page is the centerpiece recruiters will see first.

1. **Home / search.** A ticker search box, plus a leaderboard of the most positive and most negative calls from the latest earnings season.
2. **Ticker page.** An interactive price chart (2 to 5 years) with a marker on each earnings date. Each marker is colored green to red by call sentiment and sized by the next-day move. Hovering shows the quarter, sentiment score, predicted move and actual move.
3. **Call detail.** Opened by clicking a marker. It shows:
    - A sentiment timeline across the call, so you can see where tone shifts (often when the Q&A starts).
    - A CEO vs. CFO vs. analysts comparison bar.
    - The transcript with each sentence highlighted by sentiment; filters for "most negative" and "forward guidance only".
    - Change vs. last quarter: did management sound more or less confident?
4. **Model results.** Backtest charts: predicted vs. actual moves, accuracy by sector, and a feature-importance chart. This page shows you evaluated the model honestly.

A stretch screen lets users paste any text (a headline, a paragraph from a call) and get a live sentiment score.

Architecture

The system splits into an offline pipeline that does all the ML and a lightweight online app that only reads its outputs.



system architecture · offline pipeline and online app

Re-running the offline pipeline (manually each quarter, or via a scheduled GitHub Actions job) refreshes the tables; the app picks them up on its next deploy.

Data sources (all free)

Use a pre-built transcript dataset for history and the SEC's free API for fresh earnings releases, so nothing depends on scraping.

Data	Source	Cost and limits	Use it for
Historical transcripts	kurry/sp500_earnings_transcripts on Hugging Face	Free, MIT license; S&P 500 and US large-caps, 2005–2025	Main training and backtest corpus

Data	Source	Cost and limits	Use it for
Small labeled starter set	jlh-ibm/earnings_call	Free, CC0; 188 transcripts with matching stock prices	Prototype the pipeline in week 1
Recent transcripts	API Ninjas earnings transcript endpoint	Free key, no card; check the current quota	Calls from 2026 onward
Earnings press releases	SEC EDGAR submissions API (8-K, Exhibit 99.1)	Free, no key; declare a User-Agent with your email; max 10 requests/second	Exact earnings dates, plus a second text source
Daily prices	yfinance	Free, unofficial; throttles heavy use (HTTP 429)	Stock and S&P 500 (SPY) daily closes
Sentiment labels	Financial PhraseBank	Free	Evaluating or fine-tuning the sentiment model

Notes:

- Avoid scraping sites like Motley Fool or Seeking Alpha directly; their terms usually forbid it, and an interviewer may ask.
- yfinance is unofficial and Yahoo tightened rate limits in 2026. Download prices once, cache them locally, and pin the library version.
- Coverage gap: the Hugging Face dataset ends in 2025. If the API Ninjas free quota is too small, the app can show 2026 quarters using 8-K press-release text instead of transcripts.

ML approach

The ML has two stages: a transformer scores sentence sentiment, then a gradient-boosted model turns call-level features into a return prediction.

Stage 1: sentence sentiment

- Split each transcript into sentences and tag each with section (prepared remarks or Q&A) and speaker role (CEO, CFO, other executive, analyst, operator). Drop operator lines.
- Score each sentence with ProsusAI/finbert, which outputs probabilities for positive, negative and neutral. Sentence score = P(positive) − P(negative).
- Run scoring offline in batches (Google Colab or Kaggle free GPUs), not in the web app.
- Stretch: fine-tune FinBERT on a few hundred hand-labeled earnings-call sentences and compare accuracy against the base model. This is a strong interview story.

Stage 2: call-level features

- Mean sentiment for the whole call, prepared remarks, and Q&A separately.
- Gap between prepared-remarks and Q&A tone (executives scripted vs. under questioning).
- CEO vs. CFO sentiment; analyst question negativity.
- Share of negative sentences; tone of forward-looking sentences (contain "expect", "guidance", "outlook").
- Change in each metric vs. the company's previous call.
- Controls: prior 20-day return, 20-day volatility, market cap bucket, sector.
- Stretch: earnings surprise (actual vs. estimated EPS) to test whether tone adds information beyond the numbers.

Stage 3: prediction target

The main target is the abnormal return from the close before the call to 1, 3 and 5 trading days after, measured against SPY.

```
AR_{t \to t+k} = \frac{P_{t+k}}{P_{t}} - \frac{SPY_{t+k}}{SPY_{t}}
```

Calls after the close use that day's close as the start price; calls before the open use the prior day's close. Train two models: a regressor (size of the move) and a classifier (up or down).

Models

- Baselines: always predict "up"; logistic regression on price controls only.
- Main model: LightGBM or XGBoost on all features.

- Explain predictions with SHAP values; these feed the feature-importance chart in the app.

Evaluation

- Split by time, never randomly: train 2005–2019, validate 2020–2022, test 2023–2025. A random split leaks future information.
- Metrics: directional accuracy, ROC-AUC, Spearman correlation between predicted and actual return, and the return of a simple long-top-decile / short-bottom-decile portfolio.
- Success bar: beat the price-only baseline on the test period. A small edge (for example AUC 0.53–0.56) is a realistic and credible result; say so plainly in the app.

Tech stack and free hosting

Do all heavy computation offline and ship the app with precomputed results; free hosts have about 1–3 GB of memory and sleep when idle.

Layer	Choice	Why
Language	Python 3.11+	One language across pipeline, model and app
Data processing	pandas, DuckDB	DuckDB queries Parquet files fast with no database server
NLP	Hugging Face `transformers`, `datasets`; FinBERT	Free pre-trained models and dataset loading
Offline compute	Google Colab or Kaggle notebooks (free GPU)	Scoring millions of sentences is too slow on a laptop CPU
Prediction model	LightGBM, scikit-learn, SHAP	Fast, strong on tabular features, explainable
Charts	Plotly	Interactive hover, zoom and click events
App (v1)	Streamlit	Fastest path to a polished interactive app in Python
App (v2, optional)	React + Recharts front end, FastAPI back end	Stronger front-end signal; do this only after v1 works

Layer	Choice	Why
Experiment tracking	MLflow (local) or a results CSV in the repo	Shows disciplined model comparison
CI	GitHub Actions	Free for public repos; run tests and a lint check on each push

Hosting options

- **Streamlit Community Cloud (recommended for v1):** free, deploys from a public GitHub repo, unlimited public apps. Limits: roughly 1–2.7 GB memory, and apps sleep after about 12 hours without traffic. Keep FinBERT out of the deployed app so it fits.
- **Hugging Face Spaces (alternative):** the free CPU Basic tier gives 2 vCPU and 16 GB RAM, enough to run FinBERT live for the paste-text feature. Spaces sleep after 48 hours idle, and users reported new free-tier quota errors in 2026, so test it before relying on it.

Add a "cold start" note on the landing page, since a sleeping app takes 30–60 seconds to wake.

Sources: Streamlit Community Cloud limits, Hugging Face Spaces pricing, Hugging Face forum on 2026 CPU quota.

Data model

Four Parquet tables, queried with DuckDB, hold everything the app needs; together they should stay under a few hundred MB for the S&P 500.

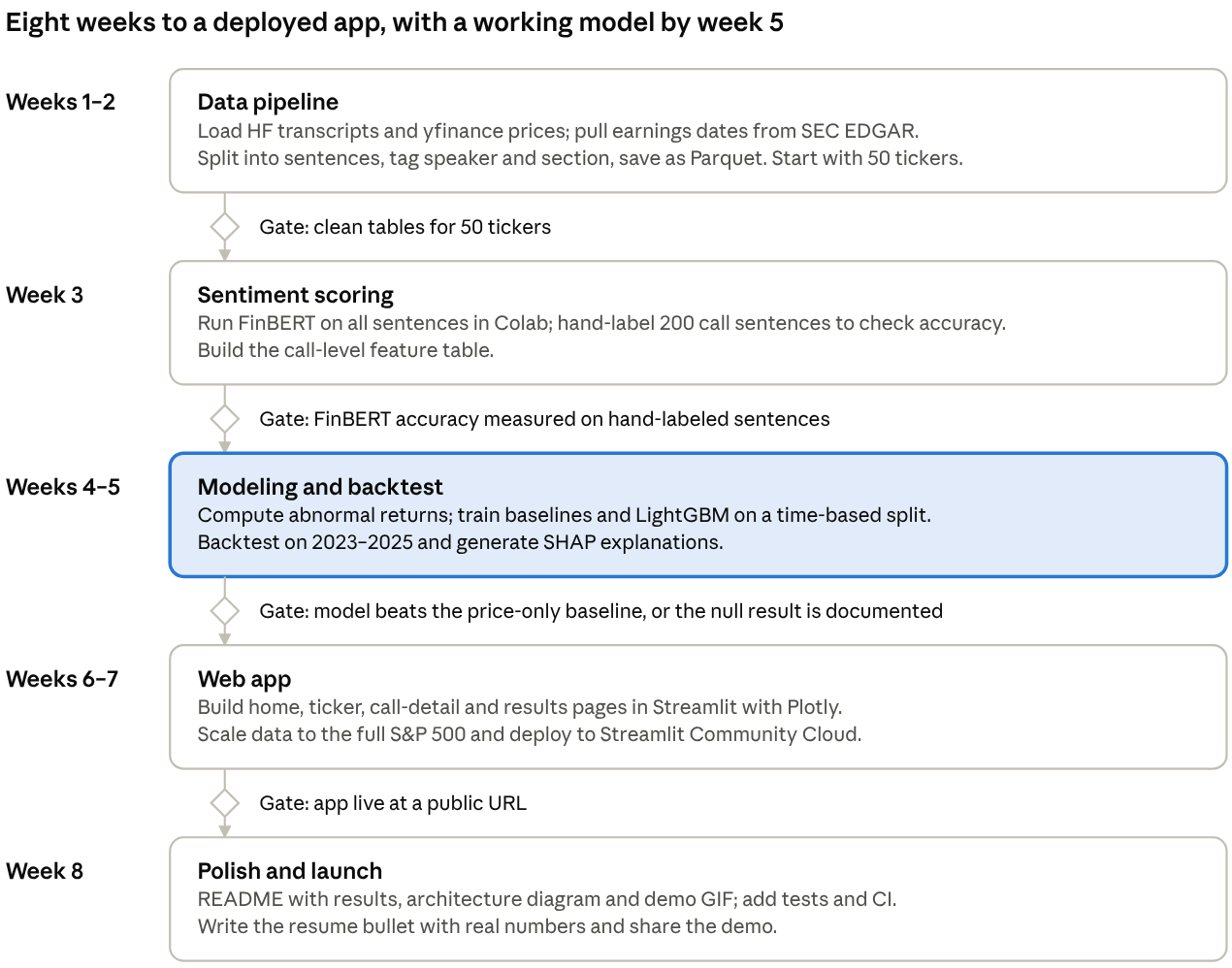
Table	Key	Main columns
`calls`	`call_id`	ticker, company, sector, fiscal_quarter, call_datetime, timing (before open / after close), source
`sentences`	`call_id`, `sentence_idx`	text, section, speaker_name, speaker_role, p_pos, p_neg, p_neu, score, is_forward_looking
`call_features`	`call_id`	all Stage 2 features, plus ar_1d, ar_3d, ar_5d (actual), pred_ar_1d, pred_up_prob, split (train / val / test)

Table	Key	Main columns
`prices`	`ticker`, `date`	adj_close, volume; includes SPY

Keep raw downloads in a separate `data/raw/` folder (not committed to Git) and commit only the small processed tables the app reads. If the `sentences` table gets too large for the free host, ship only the most recent 8 quarters of sentences and keep the full history for model training.

Milestones

The plan takes about 8 weeks at 6–8 hours a week, with a working end-to-end model by week 5.



milestone plan · 5 phases over 8 weeks

Each gate is a checkpoint: don't start the next phase until it passes. If time runs short, cut the stretch goals and the v2 React front end, not the evaluation work.

Risks, limitations, and stretch goals

The biggest risk is a model with no real edge; plan to present an honest, well-tested null or small result as a valid finding.

Risk	Impact	Mitigation
Sentiment has little predictive power	Weak headline result	Compare against baselines; frame the project as "does tone add information?" A careful answer is still impressive
Look-ahead bias (using data not known at call time)	Inflated, invalid accuracy	Time-based splits; features built only from data before the call; double-check call timing
FinBERT was trained on news, not spoken calls	Mislabels hedged or sarcastic speech	Hand-label 200–300 call sentences to measure accuracy; fine-tune if needed
yfinance breaks or throttles	Pipeline fails	Download once, cache to Parquet, pin the version; Stooq CSVs as a fallback
Transcript dataset ends in 2025	No recent quarters	API Ninjas free tier or SEC 8-K press-release text for 2026
Free host memory limits	App crashes	Precompute everything; load only the selected ticker's data
Survivorship bias (only current S&P 500 members)	Overstated results	Note it as a known limitation in the results page

Stretch goals

- Fine-tune FinBERT on your own labeled earnings-call sentences.
- Use an LLM (via a free tier) to summarize each call into 3 bullet points shown beside the sentiment.
- Add earnings surprise data and test whether tone matters more when results miss.

- Weekly GitHub Actions job that pulls new 8-K filings and refreshes the app automatically.
- Port the front end to React for a more custom look.

Resume bullet and talking points

Fill in the bracketed numbers once the backtest is done; never write a figure you have not measured.

Resume entry

Earnings Call Sentiment Analyzer | Python, PyTorch, Hugging Face, LightGBM, DuckDB, Streamlit | [live demo link] · [GitHub link]

- Built an NLP pipeline that scores [N]M sentences from [N]K S&P 500 earnings calls with FinBERT, tagging each by speaker role and call section.
- Trained a LightGBM model on [N] tone features to predict post-earnings abnormal returns; achieved [X] AUC vs. [Y] for a price-only baseline on an out-of-time 2023–2025 test set.
- Deployed an interactive web app where users explore any ticker's price history with sentiment-colored earnings events and sentence-level highlights.

Interview talking points

- Why you split data by time instead of randomly, and the look-ahead bugs you caught.
- What FinBERT got wrong on spoken language, and how you measured it.
- Why Q&A tone may matter more than scripted prepared remarks.
- How you designed around free-tier hosting limits (offline scoring, precomputed tables).
- What you would do with more data or compute.